

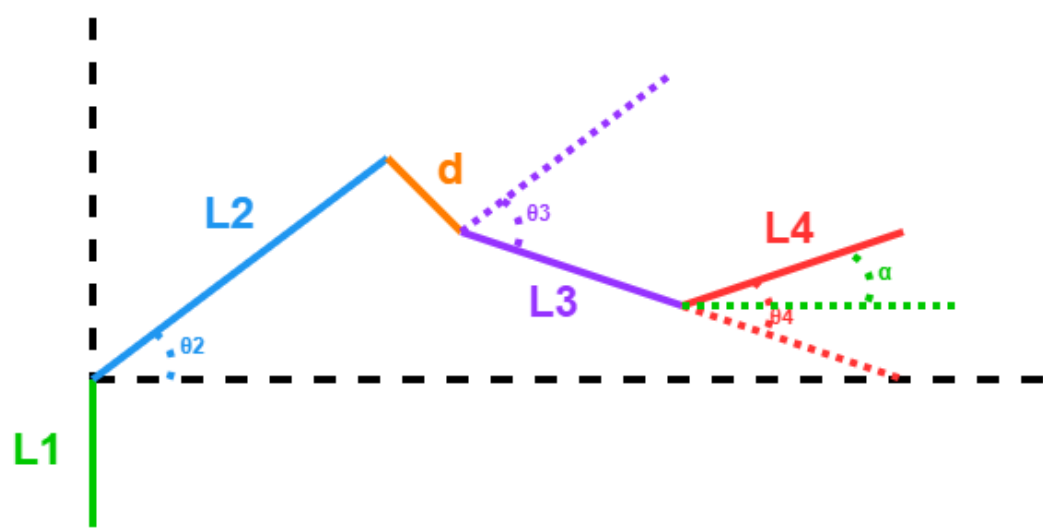
3. 运动控制教程

3.1 正运动学介绍

正运动学用于描述“关节角已知时，末端位姿如何确定”。在机械臂控制中，它主要用于当前位置恢复、姿态校验以及目标误差分析。对于该机械臂主链而言，正运动学的核心是先在该侧视工作平面内求出水平伸展量与竖直高度，再由底座回转角映射回三维空间。

3.1.1 模型定义

正运动学的全部推导都建立在统一的几何模型和角度参考系之上。如果连杆长度、偏移方向和角度参考边没有先定义清楚，后续的分量分解、向量叠加和姿态联立都会失去统一标准。连杆参数定义如下：



- L_1 ：底座到肩关节的竖直高度
- L_2 ：第二连杆主长度
- d ：第二连杆末端结构偏移
- L_3 ：第三连杆长度
- L_4 ：末端连杆长度

从几何上看，这一主链可描述为：底座到肩关节之间是一段竖直支撑 L_1 ，其后连接由主连杆 L_2 与末端偏移 d 构成的结构，偏移终点再连接第三连杆 L_3 ，最后接末端连杆 L_4 。因此，该机械结构并非理想的标准三连杆，而是一个带有偏移的串联机构。

在角度定义上， θ_2 表示 L_2 相对于水平参考线的夹角； θ_3 表示在偏移终点处、相对于一条与 L_2 平行的参考线到 L_3 的夹角； θ_4 表示从 L_3 的延长方向到 L_4 的夹角；而 α 则表示末端连杆 L_4 相对于水平参考线的绝对方向角。

由此可得各连杆方向关系为：

$$\text{dir}(L_2) = \theta_2$$

$$\text{dir}(L_3) = \theta_2 + \theta_3$$

$$\text{dir}(L_4) = \theta_2 + \theta_3 + \theta_4 = \alpha$$

需要特别注意的是， θ_3 是一个局部角（相对角），并不是第三连杆相对于水平参考线的绝对方向角，其绝对方向需通过逐级累加得到。

3.1.2 姿态关系

在主链中，末端姿态不是单独某个关节决定的，而是肩、肘、腕三个关节共同作用的结果。先写出姿态关系，后面才能区分“位置求解”和“姿态补偿”各自由哪些角承担。当末端姿态 α 已知后， θ_4 不再是独立自由量，而是由 θ_2 、 θ_3 联合决定。

末端姿态满足：

$$\alpha = \theta_2 + \theta_3 + \theta_4$$

联立可得腕部补偿角：

$$\theta_4 = \alpha - \theta_2 - \theta_3$$

3.1.3 工作平面向量分解

机械臂末端位置本质上是多个连杆位移的叠加结果。要把这个叠加过程写成可计算的公式，必须先把每一段连杆按其绝对方向角分解到水平轴和竖直轴上。

在工作平面内，末端位置可以看成若干个向量的代数和。每一根连杆都先根据其绝对方向角分解到水平轴和竖直轴，再进行逐项叠加。

第二连杆主方向向量：

$$\mathbf{p}_2 = \begin{bmatrix} L_2 \cos \theta_2 \\ L_2 \sin \theta_2 \end{bmatrix}$$

偏移项向量：

$$\mathbf{p}_{off} = \begin{bmatrix} d \sin \theta_2 \\ -d \cos \theta_2 \end{bmatrix}$$

第三连杆向量：

$$\mathbf{p}_3 = \begin{bmatrix} L_3 \cos(\theta_2 + \theta_3) \\ L_3 \sin(\theta_2 + \theta_3) \end{bmatrix}$$

第四连杆向量：

$$\mathbf{p}_4 = \begin{bmatrix} L_4 \cos(\theta_2 + \theta_3 + \theta_4) \\ L_4 \sin(\theta_2 + \theta_3 + \theta_4) \end{bmatrix}$$

其中，第三连杆和第四连杆使用的是绝对方向角，所以分别出现 $\theta_2 + \theta_3$ 与 $\theta_2 + \theta_3 + \theta_4$ 。

3.1.4 工作平面总位移

单独列出每根连杆的向量，只能说明局部几何关系；只有把各项统一相加，才能得到末端在工作平面中的整体位置表达式，也就是正运动学真正需要的位移结果。

设侧视平面中的水平伸展量为 $length$ ，竖直高度为 $height$ ，则

$$\mathbf{p}_{plane} = \begin{bmatrix} 0 \\ L_1 \end{bmatrix} + \mathbf{p}_2 + \mathbf{p}_{off} + \mathbf{p}_3 + \mathbf{p}_4$$

化简得：

$$length = L_2 \cos \theta_2 + d \sin \theta_2 + L_3 \cos(\theta_2 + \theta_3) + L_4 \cos(\theta_2 + \theta_3 + \theta_4)$$

$$height = L_1 + L_2 \sin \theta_2 - d \cos \theta_2 + L_3 \sin(\theta_2 + \theta_3) + L_4 \sin(\theta_2 + \theta_3 + \theta_4)$$

式中给出了侧视工作平面中的位置解。可以看到，偏移量 d 只在第二级结构中出现，但会同时影响水平分量和竖直分量。

3.1.5 空间映射

前述推导均基于侧视工作平面，而机械臂实际运行在三维空间中。为了得到末端执行器的真实空间坐标，需要将工作平面中的结果映射到三维坐标系中。

具体来说，工作平面中的水平伸展量通过底座回转角 θ_1 投影到空间坐标系，从而得到 x 、 y 方向的位置；平面内的高度分量则直接对应空间坐标中的 z 方向。空间坐标表达为：

$$x = length \cos(-\theta_1)$$

$$y = length \sin(-\theta_1)$$

$$z = height$$

所以机械臂的正运动学可以分为两个步骤：先在二维工作平面内求解末端的水平伸展量与高度，再通过底座回转角完成三维空间映射。正运动学最终建立如下映射关系：

$$(\theta_1, \theta_2, \theta_3, \theta_4) \rightarrow (x, y, z, \alpha)$$

3.2 逆运动学介绍

逆运动学的目标是由末端位姿反求关节角，简单来说就是已知机械臂末端的坐标，反过来求每个关节该转多少角度：

$$(x, y, z, \alpha) \rightarrow (\theta_1, \theta_2, \theta_3, \theta_4)$$

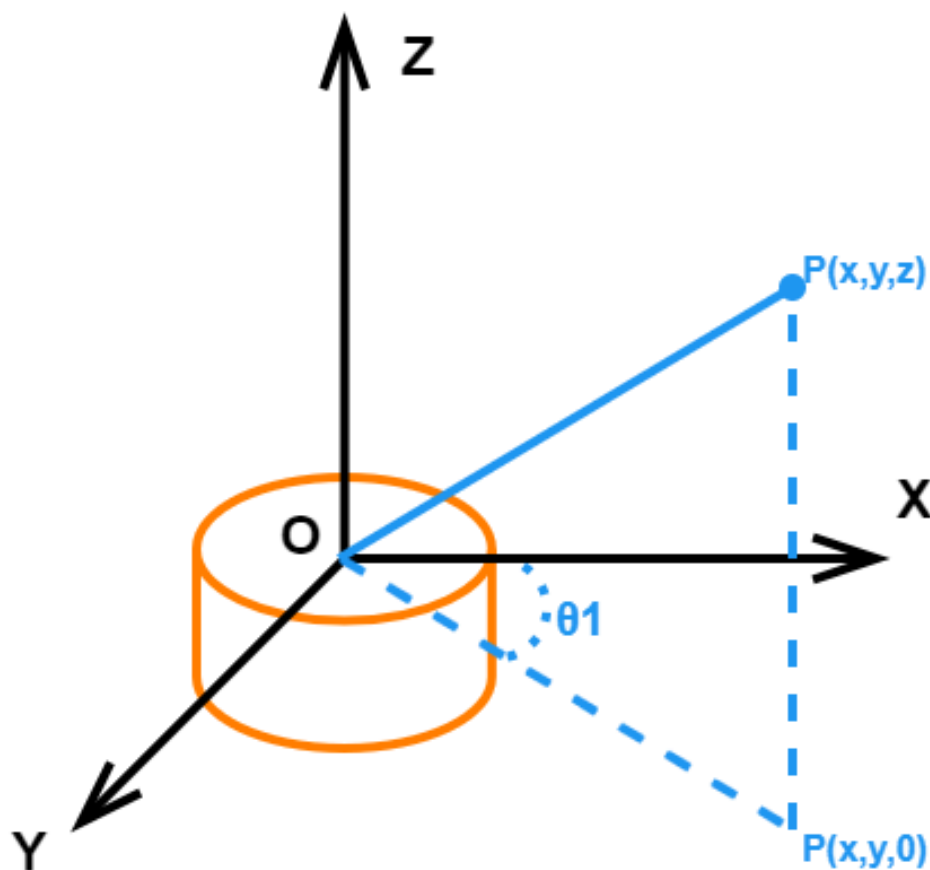
与正运动学相比，它不是逐级向前叠加，而是从末端目标反向恢复各级几何关系，因此更依赖辅助构造与角度联立。逆运动学的难点主要来自三个方面：

- 末端目标是空间坐标，而关节变量是角度，二者属于不同描述空间；
- 末端姿态 α 会同时约束 $\theta_2, \theta_3, \theta_4$ ；
- 第二级结构中存在偏移 d ，导致实际机械结构不是标准二连杆。

所以逆运动学不能直接一次写出所有关节角，而是要经过坐标降维、腕点回推、等效连杆、三角形约束和角度恢复几个阶段，逐步把空间目标转化为真实关节解。

3.2.1 坐标降维

底座回转和主链伸展在几何上属于两类不同运动。若不先将三维问题拆开，底座方位与平面连杆关系会耦合在一起，后续很难直接使用平面几何工具。



逆解首先将目标点压缩到工作平面。设目标点在水平面的投影半径为：

$$length = \sqrt{x^2 + y^2}$$

则底座角为：

$$\theta_1 = -\text{atan2}(y, x)$$

这样三维问题被分解为底座回转角 θ_1 和侧视工作平面内的其余三个角。

之所以要先做空间降维，是因为底座回转只影响目标点在水平面中的方位，而不改变主链在侧视平面中的长度关系。先把 θ_1 单独求出，就可以把剩余求解压缩为一个二维平面内的几何问题。

- 将三维耦合问题拆成“平面方位 + 平面连杆”两部分

- 后续可以直接使用腕点回推、余弦定理和反三角函数
- 避免在三维空间中同时处理底座回转与主链伸展

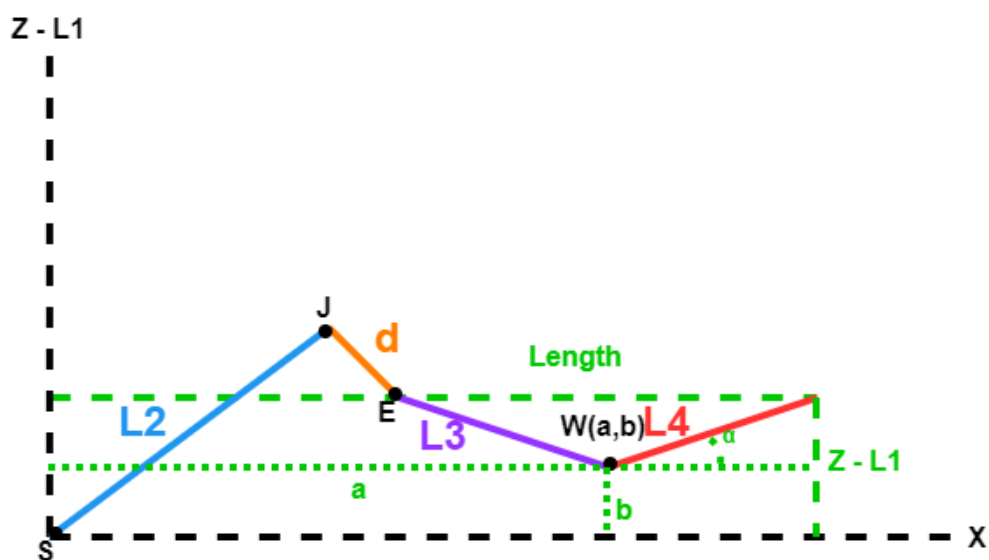
3.2.2 腕点回推

当末端姿态 α 已知时，第四连杆 L_4 的方向已经确定。先把 L_4 从整体中剥离出来，可以把“末端点到达目标”转化为“腕点到达目标”，从而减少一个直接参与几何求解的连杆。由于末端姿态 α 已知，第四连杆 L_4 的方向也随之确定，因此可先将末端点沿 L_4 反方向回推，得到腕点 (a, b) ：

$$a = length - L_4 \cos \alpha$$

$$b = z - L_1 - L_4 \sin \alpha$$

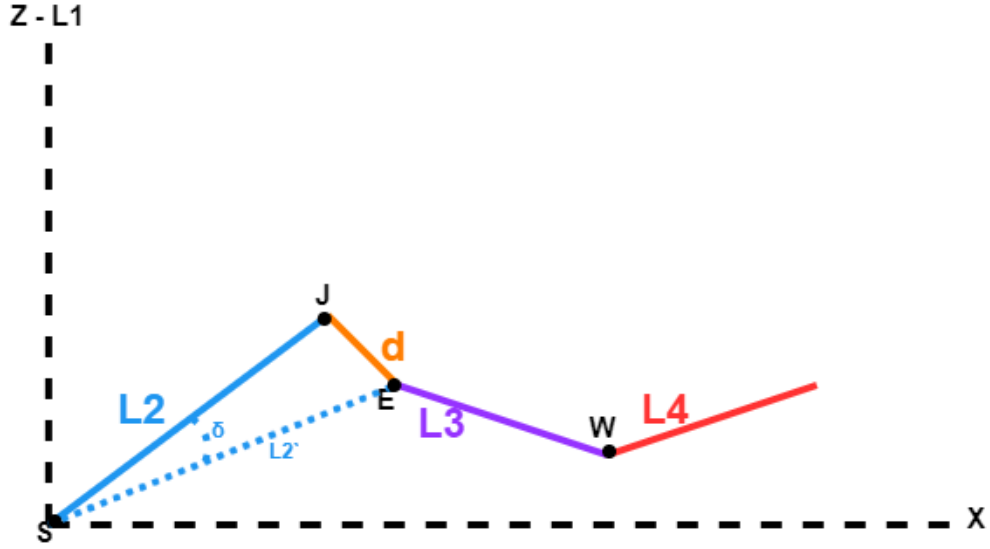
这样，逆解主问题就从“末端点到达目标”转化为“由 L_2 、偏移 d 、 L_3 组成的前级主链将腕点送到目标位置”。



3.2.3 等效连杆

由于肩关节到腕点之间并不是标准的单根第二连杆，而是“主长度 L_2 + 结构偏移 d + 第三连杆”组合出来的真实机械结构。若不先做等效化处理，就无法直接套用标准二维二连杆的求解方法。因为第二级存在结构偏移 d ，肩关节到腕点之间的真实机械结构不是标准二连杆，需先做等效化处理。设等效第二连杆长度为 L'_2 ，修正角为 δ ，所以肩关节到腕点之间的实际机械结构为：

$$S \rightarrow J \rightarrow E \rightarrow W$$



其中：

- $S \rightarrow J$ 对应主长度 L_2
- $J \rightarrow E$ 对应结构偏移 d
- $E \rightarrow W$ 对应第三连杆 L_3

由于中间多了一段偏移 d ，所以这段机械结构不能直接当作标准的“肩关节到肘关节再到腕点”的二连杆处理。

$$L'_2 = \sqrt{L_2^2 + d^2}$$

$$\delta = \arctan\left(\frac{d}{L_2}\right)$$

经过这一步后，肩关节到偏移终点的复杂机械结构可用“等效连杆 L'_2 + 修正角 δ ”表示，随后与第三连杆 L_3 组成标准二维二连杆三角形。

3.2.4 三角形约束

只要机械结构被等效为 L'_2 与 L_3 ，逆运动学就可以转化为一个标准三角形问题。只有先建立这个三角形，余弦定理和反三角函数才有明确的几何对象可用。设肩关节到腕点的距离为：

$$D = \sqrt{a^2 + b^2}$$

则得到边长为 L'_2 、 L_3 、 D 的标准三角形。后续所有角度恢复都建立在这个三角形上。设等效肘角为 γ ，则由余弦定理：

$$\cos \gamma = \frac{L_2'^2 + L_3^2 - D^2}{2L'_2L_3}$$

$$\gamma = \arccos\left(\frac{L_2'^2 + L_3^2 - D^2}{2L'_2L_3}\right)$$

$$\phi = \arccos \left(\frac{L_2'^2 + D^2 - L_3^2}{2L_2'D} \right)$$

则等效第二连杆的绝对方向角为：

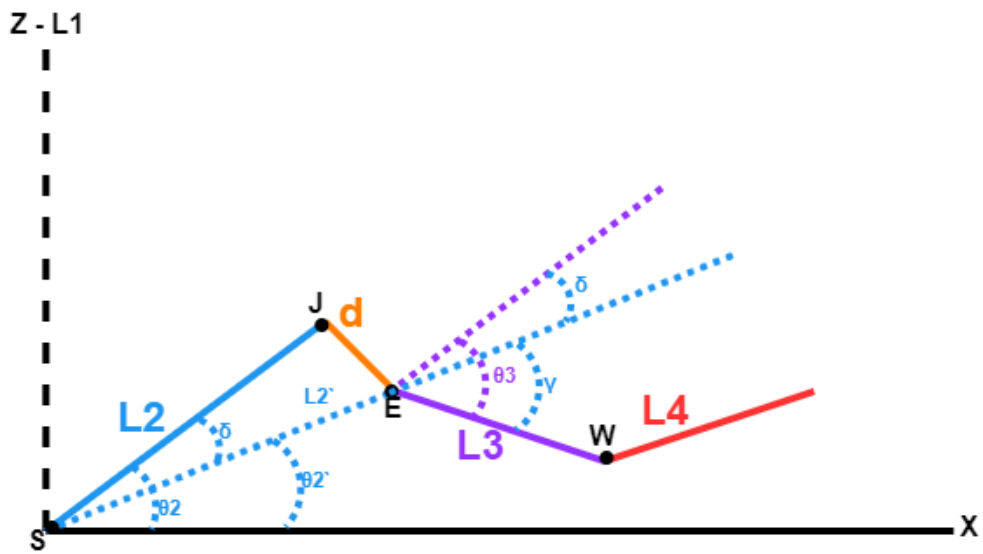
$$\theta_2' = \beta + \phi$$

联立修正角关系：

$$\theta_2 = \theta_2' + \delta$$

因此可得：

$$\theta_2 = \beta + \phi + \delta$$



第三连杆的绝对方向既可写为：

$$\theta_2 + \theta_3$$

也可写为：

$$\theta_2' + \gamma$$

联立

$$\theta_2 + \theta_3 = \theta_2' + \gamma$$

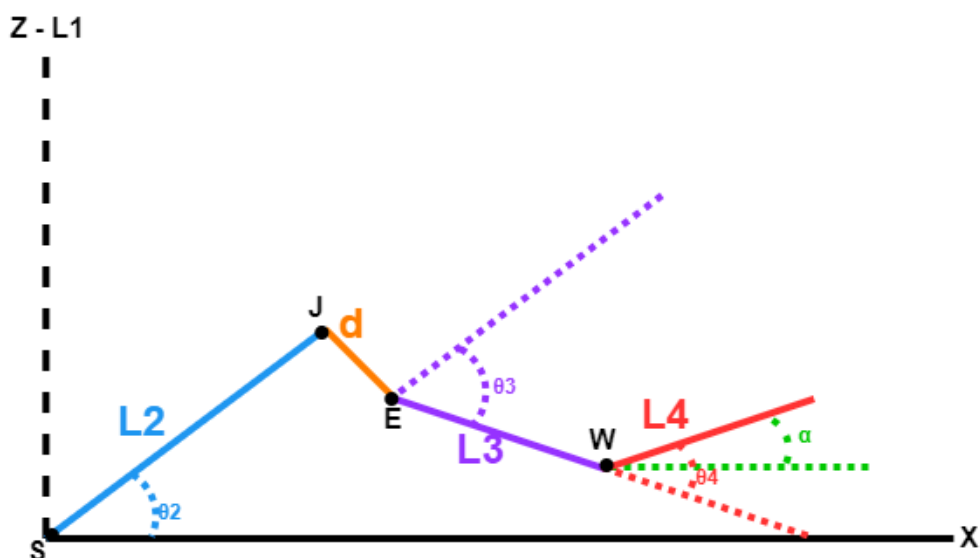
以及

$$\theta_2 = \theta_2' + \delta$$

化简得：

$$\theta_3 = \gamma - \delta$$

到这里为止， $\beta, \phi, \gamma, \delta$ 仍然只是等效三角形中的辅助几何量，还不是控制器真正需要的实际关节角。因此还必须进行角度恢复，把等效模型中的角度关系重新映射回真实机械结构中的 $\theta_2, \theta_3, \theta_4$ 。



最后由姿态约束

$$\alpha = \theta_2 + \theta_3 + \theta_4$$

联立可得：

$$\theta_4 = \alpha - \theta_2 - \theta_3$$

之所以需要角度恢复，是因为余弦定理直接给出的只是等效几何关系，而真实机械臂中还存在偏移修正角 δ 以及末端姿态约束 $\alpha = \theta_2 + \theta_3 + \theta_4$ 。只有把这些约束重新联立，才能从“等效三角形中的几何角”得到“真实机械结构中的关节角”，从而形成可执行的逆解结果。

3.3 串口协议解析

串口协议是设备之间通过串行通信方式进行数据交换时所遵循的一套规则。它通常规定了数据的发送格式、数据长度、功能标识、参数内容以及校验方式等内容。单片机、传感器、舵机驱动板、视觉模块等设备，往往都是通过串口协议来完成指令下发与数据回传。

3.3.1 设备主从通信原理

本节旨在介绍关于NexArm在与PC主机通信时的主从关系的详细信息，理解NexArm是如何作为从机与其他设备进行通信的，以及其他设备如何作为主机控制NexArm。在主从机控制系统中NexArm都是作为从机设备，通过UART串口与其他设备进行信息传输。

3.3.1.1 NexArm作为从机

1. 接收解析主机发送的信号：

等待串口信号，若串口有数据接收到，则根据通讯协议解析串口数据，可以根据数据信息调用对应的函数功能。

2. 根据接收数据调用NexArm的功能：

当解析出信号时，就需要调用对应的NexArm本设备功能，如获取固件版本、蜂鸣器控制等。

3. 数据封装与反馈：

当接收到读取命令时，则需要调用对应的读取函数后，将读取到的数据根据通讯协议封装为数据包，并发送给主机设备。

3.3.1.2 其他设备作为主机

1. 指令封装与发送：

主机需要将控制指令和数据根据通讯协议封装为数据包，发送给设备。

2. 控制协调：

主机设备需管理好整个系统的协同工作，确保NexArm和其他设备之间的通信和操作无冲突，保持良好的工作状态。

3. 数据接收：

主机读取机器人状态时，在发送读取指令后，需要接收NexArm发送的状态信息数据，确保数据的完整性和正确性，并解析数据包，提取其中的有用信息。

3.3.2 设备连接

Note

程序在下载前，请确认串口驱动已经安装。

使用USB数据线连接主机与NexArm主板上的 USB 串口。



3.3.3 数据传输格式

3.3.3.1 串口参数

NexArm的UART串口数据传输配置默认为：

波特率	1000000
数据位	8
校验位	None
停止位	1

3.3.3.2 数据帧格式

NexArm 与主机设备进行通信时，需要遵循统一的数据帧格式，只有按照约定好的协议进行数据收发与解析，双方才能正确识别和处理通信内容。若数据中出现多字节整型(int16,int32)均采用**小端模式**(低位在前)。

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	ID	Length	CMD	Parameter 1 ...Parameter N	Check

- **帧头**：连续接收2个 0xFF，表示有数据包到达。
- **标识位**：0xFF 为系统指令（控制板处理）；0x01~0x06 为透传指令（发往特定舵机）。
- **数据长度**：2 + 参数字节数（即 CMD + 参数 + Check Sum）。
- **功能号**：用于标明一个信息帧的用途。
- **参数**：表示传输的数据信息。
- **校验码**：从 ID 开始累加到最后一个参数，取低8位再取反。 $Sum = \sim(ID + Len + CMD + ParameterN) \& 0xFF$ 。

3.3.5 功能指令解析

3.3.5.1 系统基础指令

1. **固件版本查询(功能号：0x01)**：用于获取控制器的固件版本。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x02	0x01	/	0xFD

从机返回数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x03	0x01	版本号(3byte)	Check

示例：查询固件版本，若返回数据 01 00 00 表示1.0.0版本。

```
FF FF FF 02 01 FD
```

2. **电池电压查询(功能号：0x02)**：用于获取电源电压数据，单位mV。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x02	0x02	/	0xFC

从机返回数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x04	0x02	电源电压(2bytes)	Check

示例：查询电池电压，若返回数据 E0 1F 表示 0x1FE0，即 8160mV，采用小端模式。

```
FF FF FF 02 02 FC
```

3. 蜂鸣器控制(功能号：0x09)：用于控制蜂鸣器的开启时长、关闭时长、循环次数和发声频率。

主机发送数据：

帧头 1	帧头 2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x0E	0x09	开启时间(4bytes) + 关闭时间(4bytes) + 次数(2bytes) + 频率(2bytes)	Check

示例：控制蜂鸣器鸣响100ms，停100ms，循环2次，频率2000Hz。

```
FF FF FF 0E 09 64 00 00 00 64 00 00 00 02 00 D0 07 48
```

3.3.5.2 坐标与运动学控制指令

1. 坐标设置(功能号：0x08)：用于向机械臂发送末端目标坐标和姿态，由底层完成逆运动学求解。

主机发送数据：

帧头 1	帧头 2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x10	0x08	Pitch(2bytes) + X(2bytes) + Y(2bytes) + Z(2bytes) + Roll(2bytes) + Claw(2bytes) + Time(2bytes)	Check

参数说明如下：

参数名	类型	说明
Pitch	int16	末端俯仰角，单位为度
X	int16	末端 X 坐标，单位 mm
Y	int16	末端 Y 坐标，单位 mm
Z	int16	末端 Z 坐标，单位 mm
Roll	int16	末端横滚角，单位为度
Claw	int16	爪子开合角目标值，单位为度
Time	uint16	运动时间，单位 ms

示例：控制在1000ms内机械臂移动到指定位置 Pitch=0, x=200, Y=100, Z=200, Roll=0, Claw=0, Time=1000ms

```
FF FF FF 10 08 00 00 C8 00 64 00 C8 00 00 00 00 00 E8 03 09
```

2. 当前坐标查询(功能号：0x0B)：用于读取机械臂当前末端坐标信息。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x02	0x0B	/	0xF3

从机返回数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0x5A	0x08	0x0B	X(2bytes) + Y(2bytes) + Z(2bytes)	Check

示例：查询机械臂当前坐标。

```
FF FF FF 02 0B F3
```

3. 坐标增量控制(功能号：0x32)：用于在机械臂当前位置基础上做相对位移控制，并可同时控制爪子的开合增量。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x10	0x32	dX(2bytes) + dY(2bytes) + dZ(2bytes) + dPitch(2bytes) + dRoll(2bytes) + dClaw(2bytes) + Time(2bytes)	Check

参数说明如下：

参数名	类型	说明
dX	int16	X 轴增量，单位 mm
dY	int16	Y 轴增量，单位 mm
dZ	int16	Z 轴增量，单位 mm
dPitch	int16	俯仰角增量，单位为度乘10
dRoll	int16	横滚角增量，单位为度

参数名	类型	说明
dClaw	int16	爪子开合角增量，单位为度
Time	uint16	运动时间，单位 ms

示例1：仅爪子减少20度，运动时间1000ms。

```
FF FF FF 10 32 00 00 00 00 00 00 00 00 00 00 EC FF E8 03 E8
```

示例2：X轴增加10mm，同时爪子张开10度，运动时间500ms。

```
FF FF FF 10 32 0A 00 00 00 00 00 00 00 00 00 F6 FF F4 01 CA
```

3.3.5.3 无线与系统配置指令

- 1. **ESP-NOW 同步模式开关(功能号：0x21)**：用于控制机械臂是否进入实时同步控制模式，需要先手动将模式切换至ESP-NOW。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x03	0x21	switch	Check

参数说明如下：

参数名	类型	说明
switch	uint8	同步控制模式开关，0x00关闭,0x01开启

示例：开启 ESP-NOW 同步模式。

```
FF FF FF 03 21 01 DB
```

- 2. **设置全局加速度(功能号：0x1F)**：用于设置机械臂全局运动加速度。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x03	0x1F	accel(1byte)	Check

参数说明如下：

参数名	类型	说明
accel	uint8	机械臂全局加速度，范围0~254

示例：设置全局加速度为 40。

```
FF FF FF 03 1F 28 B6
```

3. 设置 ESP-NOW 通道(功能号：0x1E)：用于设置无线通信通道。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x03	0x1E	channel(1byte)	Check

参数说明如下：

参数名	类型	说明
channel	uint8	无线通信通道号，范围1~13

示例：设置 ESP-NOW 通道为 6。

```
FF FF FF 03 1E 06 D9
```

3.3.5.4 底盘与外设控制指令

1. 麦克纳姆轮控制(功能号：0x22)：用于控制底盘前后、横移和旋转速度。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x05	0x22	Vx(1byte) + Vy(1byte) + Vz(1byte)	Check

参数说明如下：

参数名	类型	说明
Vx	int8	前后速度，取值范围-100 ~ 100
Vy	int8	左右平移速度，取值范围-100 ~ 100
Vz	int8	旋转速度，取值范围-100 ~ 100

示例：控制底盘以前进速度 30、侧移 0、旋转 0 运行。

FF FF FF 05 22 1E 00 00 BB

2. 步进电机控制(功能号：0x13)：用于控制步进电机运行目标步数。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x06	0x13	step(4bytes)	Check

参数说明如下：

参数名	类型	说明
step	int32	目标脉冲数，正负用于决定方向

示例：控制步进电机运行 1000 步。

FF FF FF 06 13 E8 03 00 00 FC

3.3.5.5 单舵机寄存器控制

当标识位 ID = 0x01 ~ 0x06 时，表示对指定舵机进行透传控制。

1. 舵机力矩控制(写寄存器：0x03，寄存器地址：0x28)：用于控制舵机上电/掉电，释放力矩后可随意掰动。

主机发送数据：

帧头1	帧头2	标识位	数据长度	写寄存器	寄存器地址	参数	校验码
0xFF	0xFF	ID	0x04	0x03	0x28	value(1byte)	Check

参数说明如下：

参数名	类型	说明
value	uint8	力矩开关参数，0x00表示释放，0x01表示锁死

示例：控制 1 号舵机打开力矩。

FF FF 01 04 03 28 01 CE

2. 舵机加速度控制(写寄存器：0x03，寄存器地址：0x29)：用于控制舵机加速度，调整舵机加减速度变速。

主机发送数据：

帧头1	帧头2	标识位	数据长度	写寄存器	寄存器地址	参数	校验码
0xFF	0xFF	ID	0x04	0x03	0x29	accel(1byte)	Check

参数说明如下：

参数名	类型	说明
accel	uint8	舵机加速度，调节范围0~254

示例：控制 1 号舵机加速度设置为 10。

```
FF FF 01 04 03 29 0A C4
```

3. 舵机位置控制(写寄存器：0x03，寄存器地址：0x2A)：用于控制舵机转动至目标位置。

主机发送数据：

帧头1	帧头2	标识位	数据长度	写寄存器	寄存器地址	参数	校验码
0xFF	0xFF	ID	0x05	0x03	0x2A	pos(2byte)	Check

参数说明如下：

参数名	类型	说明
pos	uint16	舵机位置参数，范围0-4096

示例：控制 1 号舵机转到位置 2048。

```
FF FF 01 05 03 2A 00 08 C4
```

4. 舵机速度控制(写寄存器：0x03，寄存器地址：0x2E)：用于控制舵机转动时的运行步数。

主机发送数据：

帧头1	帧头2	标识位	数据长度	写寄存器	寄存器地址	参数	校验码
0xFF	0xFF	ID	0x05	0x03	0x2E	spd(2byte)	Check

参数说明如下：

参数名	类型	说明
spd	uint16	每秒内的运行步数，范围0~3400

示例：控制 1 号舵机速度设置为 1000。

```
FF FF 01 05 03 2E E8 03 DD
```

5. 连续寄存器写控制(写寄存器：0x03，起始地址：0x29)：

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	ID	0x0A	0x03	ADDR(1byte) + ACC(1byte) + POS(2bytes) + RES(2bytes) + SPD(2bytes)	Check

参数说明如下：

参数名	类型	说明
ADDR	uint8	起始寄存器地址，固定为0x29
ACC	uint8	舵机加速度，调节范围0~254
POS	uint16	舵机位置参数，范围0-4096
RES	uint16	保留字节，固定填 0x00（占 2 字节）
SPD	uint16	每秒内的运行步数，范围0~3400

示例：控制 1 号舵机转到位置 2048，加速度 0，速度 0。

```
FF FF 01 0A 03 29 00 00 08 00 00 00 00 C0
```

3.3.5.6 动作组指令

1. 运行动作组(功能号：0x03)：参数为动作组编号。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x03	0x03	actions(1byte)	Check

参数说明如下：

参数名	类型	说明
actions	uint8	动作组编号

示例：运行 1 号动作组。

FF FF FF 03 03 01 F9

2. 停止动作组(功能号：0x04)：用于打断动作组运行。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x02	0x04	/	0xFA

示例：停止当前动作组运行。

FF FF FF 02 04 FA

3. 擦除动作组(功能号：0x17)：用于擦除指定编号的动作组。

主机发送数据：

帧头1	帧头2	标识位	数据长度	功能号	参数	校验码
0xFF	0xFF	0xFF	0x03	0x17	actions(1byte)	Check

参数说明如下：

参数名	类型	说明
actions	uint8	动作组编号

示例：擦除 1 号动作组。

FF FF FF 03 17 01 E5

3.3.5.7 校验码计算示例

以“运行 1 号动作组”为例，校验码计算方式如下：

```
uint8_t id = 0xFF;
uint8_t len = 0x03;
uint8_t cmd = 0x03;
uint8_t arg0 = 0x01;

uint8_t checksum = ~(id + len + cmd + arg0) & 0xFF;
// 结果为 0xF9
```

3.4 程序下载说明（必看）

机械臂在进行单独的控制功能前，需要先下载对应的控制程序，程序下载可参考以下步骤：

1. 使用Type-C数据线将NexArm与电脑进行连接。



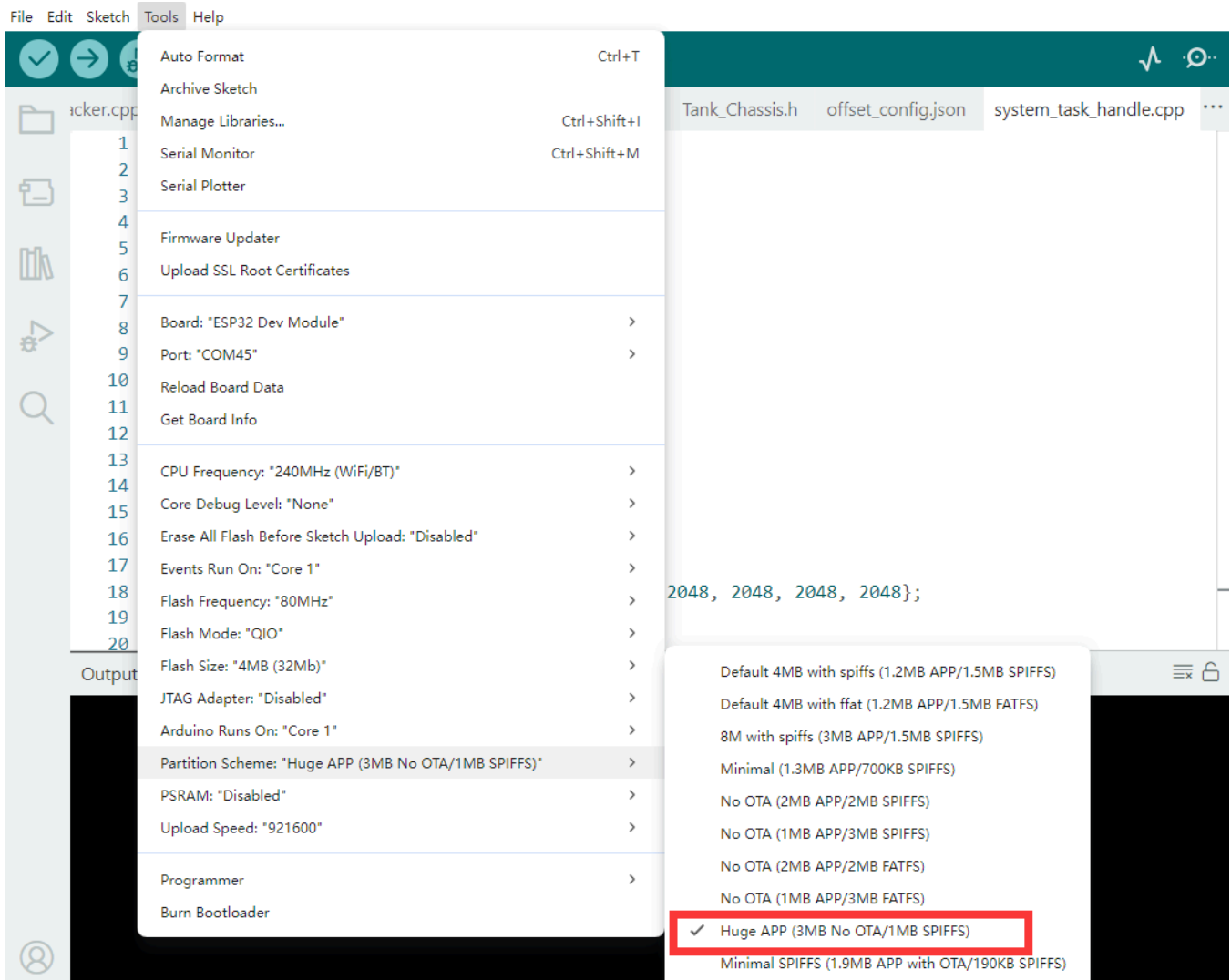
2. 打开本文档同路径下"源码"中对应的ino程序文件。

 Nex_Arm.ino

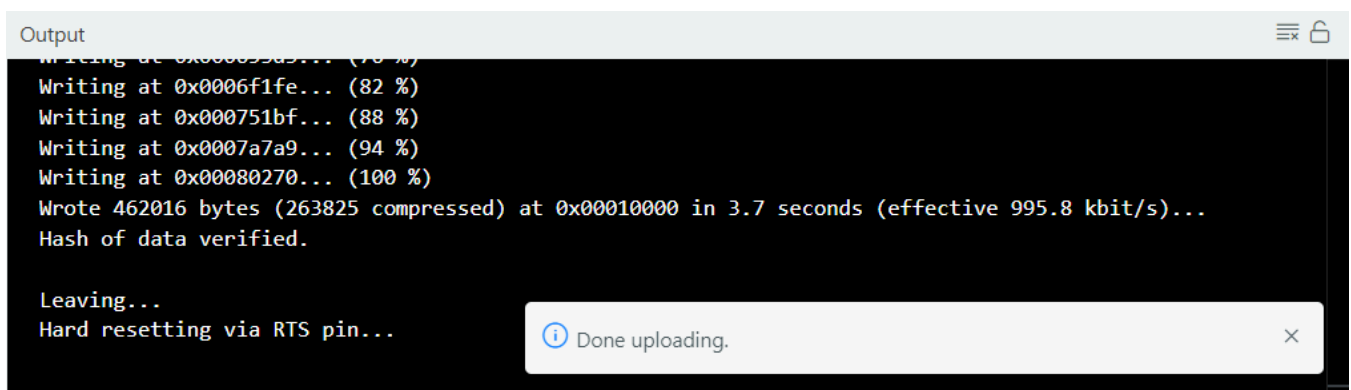
3. 打开后，选择开发板型号，具体型号如下图所示：



4. 点击菜单栏的"工具"，按照下图选择相应的ESP32的开发板配置：



5. 先点击“编译”，再点击“上传”，当软件下方输出框出现如下界面，说明程序下载成功：

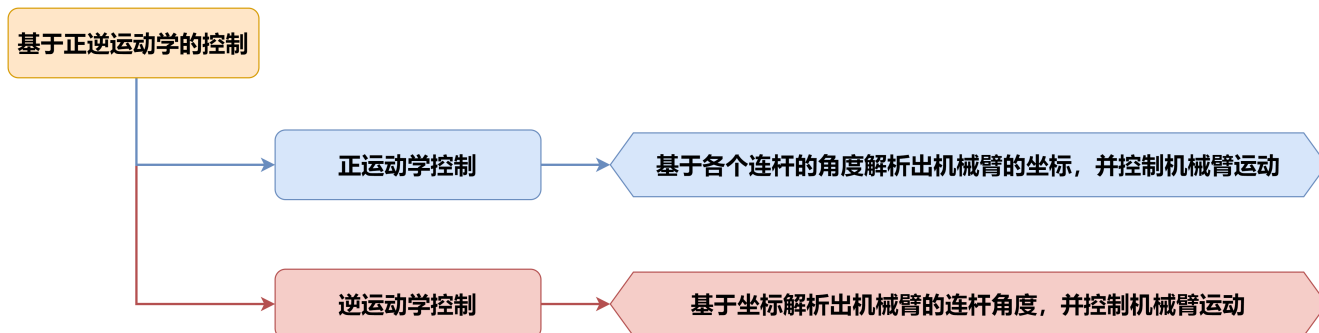


3.5 基于正逆运动学的控制

3.5.1 项目介绍

本节通过按键切换逆运动学控制和正运动学计算两种模式，用于演示机械臂末端位姿与关节角之间的转换关系。程序中一方面给出多组末端位姿，直接驱动机械臂运动；另一方面给出多组关节角数据，请求下位机进行正运动学计算，并根据计算结果再次控制机械臂运动。

3.5.2 实现流程



3.5.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“基于正逆运动学的控制”程序进行下载，具体下载步骤可参考[3.4 程序下载说明\(必看\)](#)。

3.5.4 实现效果

机械臂上电初始化完成后，按下 `BOOT` 键，机械臂在多组逆运动学目标位姿之间切换运动；按下 `USER` 键后，串口先输出正运动学计算得到的末端位姿，再根据该结果驱动机械臂运动到对应位置。OLED 会实时显示当前末端位姿信息，以便观察运动结果。

3.5.5 程序分析

1. 主程序首先定义协议解析对象、OLED 刷新时间、电池刷新时间、状态同步时间、按键使能时间，以及当前逆运动学预设索引、当前正运动学预设索引和 `fk_move_pending` 标志。

```
CommProtocol_t at32_protocol;  
uint32_t last_oled_refresh_ms = 0;  
uint32_t last_bat_refresh_ms = 0;  
uint32_t last_status_sync_ms = 0;  
uint32_t button_enable_ms = 0;  
uint8_t current_pose_index = 0;  
uint8_t current_fk_index = 0;  
bool fk_move_pending = false;
```

2. 紧接着定义了两组预设数据。 `kDemoPoses[]` 里保存的是末端位姿和运行时间，按下 `BOOT` 键时就从这里取出一组目标做逆运动学运动； `kDemoJoints[]` 里保存的是关节角、`roll` 和 `claw` 参数，按下 `USER` 键时就从这里取出一组关节角数据做正运动学计算。

```
const IkPose kDemoPoses[] = {  
    {200.0f, 0.0f, 200.0f, 0.0f, 0.0f, 0.0f, 1200},  
    {220.0f, 80.0f, 180.0f, -10.0f, 0.0f, 15.0f, 1200},  
};
```

3. 初始化中先初始化调试串口、总线舵机串口和机械臂对象，再打开舵机力矩并让机械臂回到初始位，随后启动协议解析器并注册回调函数。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
```

```

{
    (void)event_loop;

    setCpuFrequencyMhz(240);
    Serial.begin(115200);

    servo.begin(Serial1, 1000000, 16, 17);
    arm.begin();
    arm.set_torque(true);
    delay(1000);
    arm.reset_all(1500);

    at32_protocol.begin();
    at32_protocol.register_success_callback(at32_packet_callback);
}

```

4. `apply_demo_pose()` 把逆运动学数据串成了一次完整动作。函数先根据索引从 `kDemoPoses[]` 中取出当前位姿，再把坐标、姿态、夹爪和时间打印到串口，最后调用 `arm.move(...)` 把目标位姿发给下位机执行。因此按下 `BOOT` 键时，看到的的就是“读取当前目标 -> 打印参数 -> 执行运动”这一条清晰流程。

```

void apply_demo_pose(uint8_t index)
{
    const IkPose& pose = kDemoPoses[index % (sizeof(kDemoPoses) / sizeof(kDemoPoses[0]))];
    Serial.printf(
        "[IK] target pose index %u -> x=%.1f y=%.1f z=%.1f pitch=%.1f roll=%.1f claw=%.1f\n",
        index,
        pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw, pose.duration_ms
    );
    arm.move(pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw, pose.duration_ms);
}

```

5. `request_fk_demo()` 则对应正运动学，先从 `kDemoJoints[]` 中取出当前关节角数据，保存到 `last_requested_fk` 中，再把 `fk_move_pending` 置为 `true`，表示“这次只要收到 FK 结果，就继续按结果做一次真实运动”。最后函数调用 `arm.request_fk_calc(...)` 把关节角发送给下位机做正运动学计算。

```

void request_fk_demo(uint8_t index)
{
    const FkJoints& joints = kDemoJoints[index % (sizeof(kDemoJoints) /
sizeof(kDemoJoints[0]))];
    last_requested_fk = joints;
    fk_move_pending = true;
    arm.request_fk_calc(joints.j1, joints.j2, joints.j3, joints.j4, joints.roll,
joints.claw);
}

```

6. 请求发出后，后面的关键就是回包解析。`parse_pose_from_packet()` 负责把 AT32 回包里的原始字节拆成 `x`、`y`、`z`、`pitch`、`roll`、`claw`，`update_pose_from_packet()` 则把这些值同步到 `arm.current_pose`，并在 `CMD_GET_CUR_COORDS` 回包里继续提取 6 路舵机位置。这样无论是串口输出还是 OLED 刷新，拿到的都是刚刚解析完成的最新位姿。


```

void update_pose_from_packet(PacketTypeDef* rx_packet)
{
    parse_pose_from_packet(rx_packet, arm.current_pose);

    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length >= 26)
    {
        for (int i = 0; i < 6; ++i) {
            int idx = 12 + i * 2;
            last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        last_servo_feedback_ms = millis();
    }
}

```

7. `at32_packet_callback()` 函数中收到 `CMD_GET_CUR_COORDS` 或 `CMD_IKINE_RESULT_GET` 时，函数会先更新当前位姿；收到 `CMD_FKINE_RESULT_GET` 时，会先解析出 FK 计算得到的末端位姿，再检查 `fk_move_pending` 是否置位，如果已经置位，就立刻调用 `arm.move(...)` 按计算结果做一次实际运动。

```

case CMD_FKINE_RESULT_GET:
{
    ArmPose_t fk_pose = {};
    parse_pose_from_packet(rx_packet, fk_pose);

    if (fk_move_pending) {
        fk_move_pending = false;
        arm.move(fk_pose.x, fk_pose.y, fk_pose.z, fk_pose.pitch, fk_pose.roll,
fk_pose.claw, 1200);
    }
    break;
}

```

8. 最后的循环中一边持续把串口字节送入协议解析器，一边刷新按键、蜂鸣器、电池、当前状态和 OLED；当检测到 `BOOT` 键点击时调用 `apply_demo_pose()`，检测到 `USER` 键点击时调用 `request_fk_demo()`。

```

if (now >= button_enable_ms && arm.board.button.is_clicked(0)) {
    apply_demo_pose(current_pose_index);
}

if (now >= button_enable_ms && arm.board.button.is_clicked(1)) {
    request_fk_demo(current_fk_index);
}

if (now - last_status_sync_ms >= STATUS_SYNC_INTERVAL_MS) {
    last_status_sync_ms = now;
    arm.update_status();
}

if (now - last_oled_refresh_ms >= OLED_REFRESH_INTERVAL_MS) {
    last_oled_refresh_ms = now;
    arm.board.oled.show_status(

```

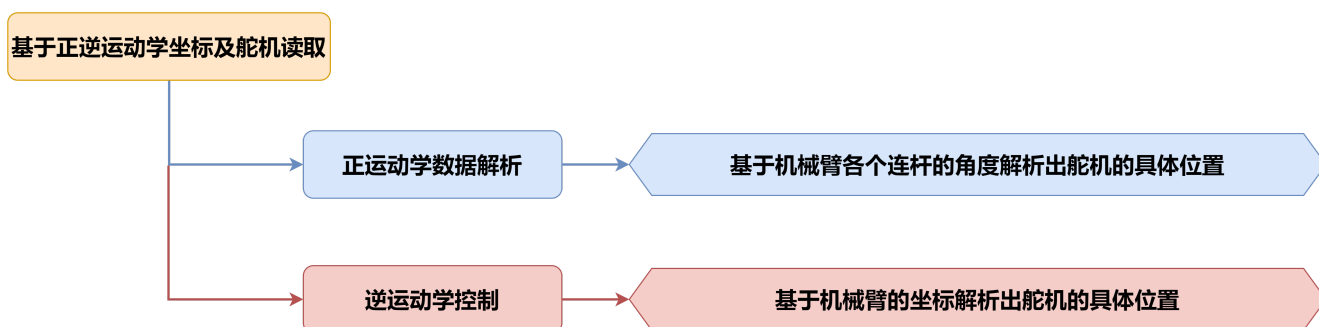
```
    arm.current_pose.x,  
    arm.current_pose.y,  
    arm.current_pose.z,  
    arm.current_pose.pitch,  
    arm.current_pose.roll,  
    arm.current_pose.claw  
};
```

3.6 基于正逆运动学坐标及舵机读取

3.6.1 项目介绍

本节通过按键读取逆运动学和正运动学的计算结果，并同时输出 6 路舵机位置数据，适合在不实际驱动机械臂动作的情况下检查运动学计算结果是否正确。

3.6.2 实现流程



3.6.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“基于正逆运动学坐标及舵机读取”程序进行下载，具体下载步骤可参考[3.4 程序下载说明\(必看\)](#)。

3.6.4 实现效果

机械臂上电后，按下 **BOOT** 键，串口输出逆运动学计算得到的末端位姿和 6 路舵机值；按下 **USER** 键后，串口输出正运动学计算得到的位姿和对应舵机值。OLED 持续显示当前末端位姿。

3.6.5 程序分析

1. 主程序先定义协议解析对象、OLED 刷新时间、电池刷新时间、按键使能时间、最近一次舵机反馈时间，以及当前 IK 读取索引、FK 读取索引。这些变量并不直接控制机械臂运动，而是控制“什么时候允许发起读取”、“当前读取的是哪组预设数据”和“界面多久刷新一次”。

```
CommProtocol_t at32_protocol;
uint32_t last_oled_refresh_ms = 0;
uint32_t last_bat_refresh_ms = 0;
uint32_t button_enable_ms = 0;
uint32_t last_servo_feedback_ms = 0;
uint8_t current_ik_index = 0;
uint8_t current_fk_index = 0;
```

2. `kIkReadTargets[]` 用于保存末端位姿数据，供逆运动学读取使用；`kFkReadTargets[]` 用于保存关节角数据，供正运动学读取使用。

```
const IkPose kIkReadTargets[] = {
    {200.0f, 0.0f, 200.0f, 0.0f, 0.0f, 0.0f, 1200},
    {220.0f, 80.0f, 180.0f, -10.0f, 0.0f, 15.0f, 1200},
};
```

3. 初始化函数中初始化串口、总线舵机串口、机械臂对象和协议解析器，然后同步一次当前状态，再清掉上电阶段可能残留的按键点击。这样后面真正按下 `BOOT` 或 `USER` 时，请求和回包都可以在一个干净的状态里进行。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    setCpuFrequencyMhz(240);
    Serial.begin(115200);

    servo.begin(Serial1, 1000000, 16, 17);
    arm.begin();

    at32_protocol.begin();
    at32_protocol.register_success_callback(at32_packet_callback);
}
```

4. `request_ik_read()` 函数从 `kIkReadTargets[]` 中取出一组末端位姿，再用 `arm.move(..., true)` 把它发送给下位机，其中最后的 `true` 表示只计算、不实际运动；`request_fk_read()` 则把一组关节角交给 `arm.request_fk_calc(...)`，只等待下位机返回正运动学结果。

```
arm.move(pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw, pose.duration_ms,
true);
```

5. 当请求发出去之后，回包里的数据需要分两步拆开。`parse_pose_from_packet()` 负责把字节流还原成 `x`、`y`、`z`、`pitch`、`roll`、`claw`，`parse_servo_positions_from_packet()` 负责继续从后半段数据里取出 6 路舵机值，`print_servo_positions()` 则把这 6 个值格式化后直接打印出来。

```

bool parse_servo_positions_from_packet(PacketTypeDef* rx_packet, int16_t servo_pos[6])
{
    if (rx_packet->elements.length < 26) {
        return false;
    }

    for (int i = 0; i < 6; ++i) {
        int idx = 12 + i * 2;
        servo_pos[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) | rx_packet->elements.args[idx]);
    }
    return true;
}

```

6. 在解析的基础上，`update_cached_feedback()` 负责把最近一次收到的实时反馈保存下来，`at32_packet_callback()` 则负责按命令类型分别处理回包。收到 `CMD_GET_CUR_COORDS` 指令时更新缓存；收到 `CMD_IKINE_RESULT_GET` 或 `CMD_FKINE_RESULT_GET` 指令时先解析位姿，再解析舵机值，最后统一打印结果。

```

if (rx_packet->elements.cmd == CMD_FKINE_RESULT_GET) {
    ArmPose_t pose = {};
    int16_t servo_pos[6] = {0};
    parse_pose_from_packet(rx_packet, pose);
    parse_servo_positions_from_packet(rx_packet, servo_pos);
    arm.current_pose = pose;

    Serial.printf(
        "[FK] result -> x=%.1f y=%.1f z=%.1f pitch=%.1f roll=%.1f claw=%.1f\n",
        pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw
    );
    print_servo_positions(servo_pos);
}

```

7. 主循环最后不断把串口字节送入解析器，再更新按键、蜂鸣器和电池状态，然后周期性刷新 OLED；当检测到 `BOOT` 键点击时，调用 `request_ik_read()` 发起一次逆运动学读取；当检测到 `USER` 键点击时，调用 `request_fk_read()` 发起一次正运动学读取。

```

if (now >= button_enable_ms && arm.board.button.is_clicked(0)) {
    request_ik_read(current_ik_index);
}

if (now >= button_enable_ms && arm.board.button.is_clicked(1)) {
    request_fk_read(current_fk_index);
}

if (now - last_oled_refresh_ms >= OLED_REFRESH_INTERVAL_MS) {
    last_oled_refresh_ms = now;
    arm.board.oled.show_status(
        arm.current_pose.x,
        arm.current_pose.y,
        arm.current_pose.z,

```

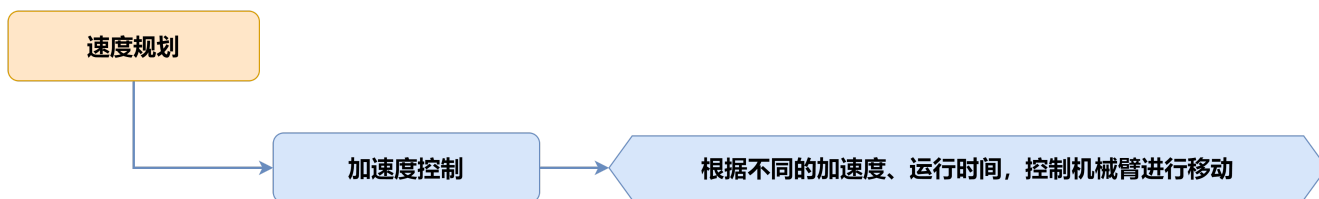
```
        arm.current_pose.pitch,  
        arm.current_pose.roll,  
        arm.current_pose.claw  
    );  
}
```

3.7 速度规划

3.7.1 项目介绍

本节课程演示机械臂在相同或相近运动任务下，分别使用不同加速度和不同运行时间进行控制。程序中既包含单段速度规划，也包含多阶段速度规划，用于观察机械臂在起步、快速移动和减速靠近阶段的运动差异。

3.7.2 实现流程



3.7.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“速度规划”程序进行下载，具体下载步骤可参考[3.4 程序下载说明\(必看\)](#)。

3.7.4 实现效果

机械臂上电后完成状态同步，默认不自动运动。按下 **BOOT** 键后，机械臂在左右两侧目标点之间切换，并依次使用 **gentle**、**balanced**、**aggressive** 三组速度规划参数运行，串口会输出当前规划名称、加速度、运行时间和目标位姿。按下 **USER** 键后，机械臂依次完成 **depart**、**cruise**、**settle** 和 **home** 四个阶段的运动，串口输出每一阶段的规划参数和目标位姿，OLED 持续显示当前末端状态。

3.7.5 程序分析

1. 主程序先定义了速度规划运行时的状态变量，包括协议解析对象、OLED 刷新时间、电池刷新时间、状态同步时间、按键使能时间、最近一次舵机反馈时间，以及当前速度规划索引和左右路径切换标志。

```
CommProtocol_t at32_protocol;  
  
uint32_t last_oled_refresh_ms = 0;  
uint32_t last_bat_refresh_ms = 0;  
uint32_t last_status_sync_ms = 0;  
uint32_t button_enable_ms = 0;  
uint32_t last_servo_feedback_ms = 0;  
  
uint8_t current_speed_index = 0;  
bool path_toggle = false;
```

2. 速度规划用到的数据被拆成了两层。PoseTarget 用来保存目标位姿，SpeedPlan 用来保存规划名称、运动加速度和运行时间；kHomePose、kLeftPose、kRightPose、kApproachPose、kSettlePose 给出了不同阶段对应的目标位姿，kSpeedPlans[] 则给出了单段规划要循环使用的三组参数。

```
struct SpeedPlan {
    const char* name;
    uint8_t move_acc;
    uint16_t duration_ms;
};

const PoseTarget kHomePose = {200.0f, 0.0f, 200.0f, 0.0f, 0.0f, 0.0f};
const PoseTarget kLeftPose = {210.0f, 90.0f, 170.0f, -8.0f, 0.0f, 10.0f};
const PoseTarget kRightPose = {210.0f, -90.0f, 170.0f, -8.0f, 0.0f, 10.0f};

const SpeedPlan kSpeedPlans[] = {
    {"gentle", 10, 2800},
    {"balanced", 35, 1600},
    {"aggressive", 75, 850},
};
```

3. 初始化阶段先初始化串口、总线舵机串口和机械臂对象，再启动协议解析器并注册 AT32 回调函数，随后同步一次当前状态并清掉上电阶段残留的按键点击。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    setCpuFrequencyMhz(240);
    Serial.begin(115200);

    servo.begin(Serial1, 1000000, 16, 17);
    arm.begin();

    at32_protocol.begin();
    at32_protocol.register_success_callback(at32_packet_callback);

    sync_arm_feedback(300);
    arm.board.button.update();
    arm.board.button.is_clicked(0);
    arm.board.button.is_clicked(1);
    button_enable_ms = millis() + BUTTON_STARTUP_GUARD_MS;
}
```

4. set_move_acc() 提供了速度规划中最关键的一层控制接口。这个函数定义在 Robot_Arm.cpp 中，调用时会把加速度值打包成 CMD_SET_MOVE_ACC 指令发送给下位机，因此主程序在控制速度时并不只是修改运行时间，而是同时把加速度参数也一起下发。

```
void Robot_Arm_t::set_move_acc(uint8_t acc)
{
    send_packet(CMD_SET_MOVE_ACC, &acc, 1);
}
```

5. `move_with_plan()` 函数先打开力矩，再根据当前 `SpeedPlan` 调用 `arm.set_move_acc(plan.move_acc)` 设置运动加速度，随后把规划名称、加速度、运行时间和目标位姿打印到串口，最后调用 `arm.move(...)` 执行运动。

```
void move_with_plan(const PoseTarget& pose, const SpeedPlan& plan)
{
    arm.set_torque(true);
    arm.set_move_acc(plan.move_acc);
    Serial.printf(
        "[Speed] plan=%s acc=%u duration=%u -> x=%.1f y=%.1f z=%.1f pitch=%.1f roll=%.1f claw=%.1f\n",
        plan.name,
        plan.move_acc,
        plan.duration_ms,
        pose.x,
        pose.y,
        pose.z,
        pose.pitch,
        pose.roll,
        pose.claw
    );
    arm.move(pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw, plan.duration_ms);
}
```

6. `run_staged_speed_demo()` 函数在内部依次定义了 `depart`、`cruise`、`settle` 三组阶段参数，然后按顺序让机械臂运动到 `kApproachPose`、`kRightPose`、`kSettlePose` 和 `kHomePose`。每切换一个阶段，都会先设置对应的加速度，再执行对应的目标位姿，因此整段动作能明显体现出起步、快速移动和减速回位的差别。

```
const SpeedPlan stage_depart = {"depart", 8, 2200};
const SpeedPlan stage_cruise = {"cruise", 70, 950};
const SpeedPlan stage_settle = {"settle", 18, 1700};

arm.set_move_acc(stage_depart.move_acc);
arm.move(kApproachPose.x, kApproachPose.y, kApproachPose.z, kApproachPose.pitch,
kApproachPose.roll, kApproachPose.claw, stage_depart.duration_ms);

arm.set_move_acc(stage_cruise.move_acc);
arm.move(kRightPose.x, kRightPose.y, kRightPose.z, kRightPose.pitch, kRightPose.roll,
kRightPose.claw, stage_cruise.duration_ms);
```

7. 速度规划动作执行期间，位姿更新仍然是通过 AT32 回包完成的。`parse_pose_from_packet()` 负责把回包字节解析成 `x`、`y`、`z`、`pitch`、`roll`、`claw`，`update_pose_from_packet()` 则把结果同步到 `arm.current_pose` 并更新反馈时间戳。这样无论机械臂当前执行哪一种规划方式，OLED 上看到的始终都是最新的末端状态。

```

void update_pose_from_packet(PacketTypeDef* rx_packet)
{
    parse_pose_from_packet(rx_packet, arm.current_pose);

    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length >= 26)
    {
        for (int i = 0; i < 6; ++i) {
            int idx = 12 + i * 2;
            last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        last_servo_feedback_ms = millis();
    }
}

```

8. 主循环中先持续处理 AT32 回包、按键更新、蜂鸣器刷新、电池刷新和 OLED 刷新；当检测到 `BOOT` 键点击时，主程序从 `kSpeedPlans[]` 中取出当前规划参数，并在左右两个目标位姿之间交替调用 `move_with_plan()`；当检测到 `USER` 键点击时，则直接调用 `run_staged_speed_demo()` 运行一整段多阶段速度规划。

```

if (now >= button_enable_ms && arm.board.button.is_clicked(0)) {
    const PoseTarget& target = path_toggle ? kLeftPose : kRightPose;
    const SpeedPlan& plan = kSpeedPlans[current_speed_index];
    move_with_plan(target, plan);

    current_speed_index = (current_speed_index + 1) % (sizeof(kSpeedPlans) /
sizeof(kSpeedPlans[0]));
    path_toggle = !path_toggle;
}

if (now >= button_enable_ms && arm.board.button.is_clicked(1)) {
    run_staged_speed_demo();
}

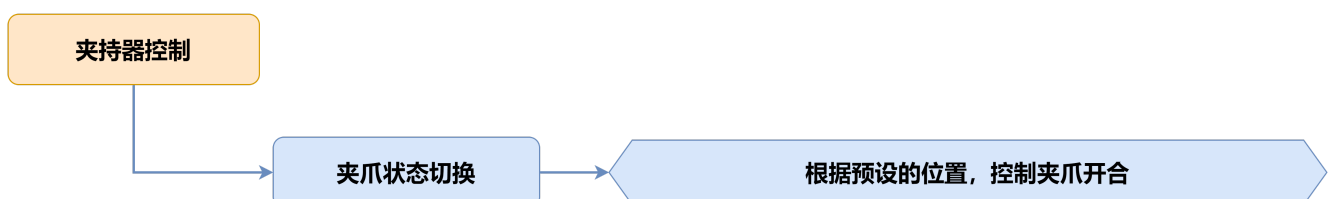
```

3.8 夹持器控制

3.8.1 项目介绍

本节课程将机械臂保持在固定待机姿态，通过改变 `claw` 参数控制夹爪张开与闭合。

3.8.2 实现流程



3.8.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“**夹持器控制**”程序进行下载，具体下载步骤可参考[3.4 程序下载说明\(必看\)](#)。

3.8.4 实现效果

机械臂上电后通过按下 **BOOT** 键，切换夹爪状态，串口输出当前夹爪角度、加速度和运行时间，同时 OLED 继续显示当前末端状态，便于观察夹爪开合过程。

3.8.5 程序分析

1. 主程序先用 **PoseTarget** 和 **GripperPreset** 把两类核心数据分开定义清楚。**PoseTarget** 表示机械臂在夹持器演示时保持不变的空间位姿，**GripperPreset** 则单独描述夹爪预设名称、夹爪角度、运动加速度和动作时间。

```
struct GripperPreset {
    const char* name;
    float claw;
    uint8_t move_acc;
    uint16_t duration_ms;
};

const PoseTarget kStandbyPose = {200.0f, 0.0f, 200.0f, 0.0f, 0.0f, 0.0f};
const GripperPreset kGripperPresets[] = {
    {"open", -50.0f, 16, 900},
    {"close", 15.0f, 16, 900},
};
```

2. 初始化阶段主要是进行串口初始化、总线舵机串口初始化、机械臂对象初始化和协议回调注册，然后同步一次当前状态，并清掉上电初期可能残留的按键状态。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    Serial.begin(115200);
    servo.begin(Serial1, 1000000, 16, 17);
    arm.begin();

    at32_protocol.begin();
    at32_protocol.register_success_callback(at32_packet_callback);
}
```

3. 当需要执行夹爪动作时，先进入 **move_to_pose_with_claw()**。该函数负责把传进来的目标位姿和夹爪角度发给下位机执行：先打开力矩，接着设置运动加速度，最后调用 **arm.move(...)** 执行动作。

```
void move_to_pose_with_claw(const PoseTarget& pose, uint8_t move_acc, uint16_t
duration_ms)
{
    arm.set_torque(true);
    arm.set_move_acc(move_acc);
    arm.move(pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw, duration_ms);
}
```

4. `apply_gripper_preset()` 把“选择预设”和“执行动作”两步接起来了。函数把当前预设里的 `claw` 值写进去，随后把预设名称、夹爪角度、加速度、运行时间和当前位姿打印到串口，最后调用 `move_to_pose_with_claw()`。

```
void apply_gripper_preset(const GripperPreset& preset)
{
    PoseTarget target = kStandbyPose;
    target.claw = preset.claw;

    Serial.printf(
        "[Gripper] preset=%s claw=%.1f acc=%u duration=%u pose=(x=%.1f y=%.1f z=%.1f
pitch=%.1f roll=%.1f)\n",
        preset.name,
        preset.claw,
        preset.move_acc,
        preset.duration_ms,
        target.x,
        target.y,
        target.z,
        target.pitch,
        target.roll
    );

    move_to_pose_with_claw(target, preset.move_acc, preset.duration_ms);
}
```

5. 夹持器动作执行之后，位姿更新仍然是通过 AT32 回包完成的。`parse_pose_from_packet()` 负责把回包中的原始字节解析成位姿字段，`update_pose_from_packet()` 再把解析结果同步到 `arm.current_pose`。

```
void update_pose_from_packet(PacketTypeDef* rx_packet)
{
    parse_pose_from_packet(rx_packet, arm.current_pose);

    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length >= 26)
    {
        last_servo_feedback_ms = millis();
    }
}
```

6. 主循环中先处理 AT32 回包、按键更新和蜂鸣器刷新，再按时间片更新电池、电流状态和 OLED；当检测到 `BOOT` 键点击时，就从 `kGripperPresets[]` 中取出当前预设并调用 `apply_gripper_preset()`，随后把 `current_gripper_index` 加一。

```
if (now >= button_enable_ms && arm.board.button.is_clicked(0)) {
    const GripperPreset& preset = kGripperPresets[current_gripper_index];
    apply_gripper_preset(preset);
    current_gripper_index = (current_gripper_index + 1) % (sizeof(kGripperPresets) /
    sizeof(kGripperPresets[0]));
}

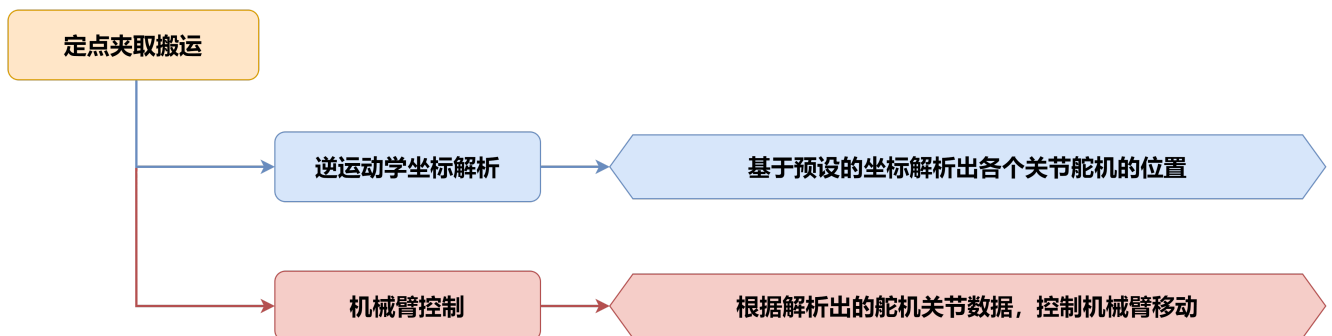
if (now - last_oled_refresh_ms >= OLED_REFRESH_INTERVAL_MS) {
    last_oled_refresh_ms = now;
    arm.board.oled.show_status(
        arm.current_pose.x,
        arm.current_pose.y,
        arm.current_pose.z,
        arm.current_pose.pitch,
        arm.current_pose.roll,
        arm.current_pose.claw
    );
}
```

3.9 定点夹取搬运

3.9.1 项目介绍

本节通过预设的一组固定点位和夹爪动作，完成机械臂的定点夹取与搬运任务。

3.9.2 实现流程



3.9.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“定点夹取搬运”程序进行下载，具体下载步骤可参考[3.4 程序下载说明\(必看\)](#)。

3.9.4 实现效果

机械臂上电后，按下 BOOT 键，机械臂按照预设流程依次执行夹取、搬运、放置和返回动作，串口会输出每一步的位姿和运行时间，OLED 持续显示当前末端状态。

3.9.5 程序分析

1. 主程序先把每个动作步骤抽象成 `PoseTarget`。结构体里统一保存 `x`、`y`、`z`、`pitch`、`roll`、`claw` 和 `duration_ms`，这样动作数据都能用同一种数据格式描述。

```
struct PoseTarget {  
    float x;  
    float y;  
    float z;  
    float pitch;  
    float roll;  
    float claw;  
    uint16_t duration_ms;  
};
```

2. `kPickPlaceSequence[]` 把动作流程真正排成了顺序表。数组中的每个元素就是一个明确的执行步骤，前几项负责下降和闭合夹爪，中间几项负责抬升和搬运，后几项负责下降、张开夹爪和回到结束位。

```
const PoseTarget kPickPlaceSequence[] = {  
    {250.0f, 0.0f, 20.0f, -90.0f, 0.0f, -50.0f, 2000},  
    {250.0f, 0.0f, 20.0f, -90.0f, 0.0f, 0.0f, 2000},  
};
```

3. 初始化函数中，先初始化串口、总线舵机串口和机械臂对象，再打开力矩，延时稳定后让机械臂回到初始位，最后启动协议解析器并注册回调。

```
arm.begin();  
arm.set_torque(true);  
delay(1000);  
arm.reset_all(1500);  
  
at32_protocol.begin();  
at32_protocol.register_success_callback(at32_packet_callback);
```

4. 每走一步具体动作时，都会进入 `run_pose_step()`。这个函数先设置力矩和运动加速度，再把当前步骤编号、目标位姿、夹爪角度和运行时间打印到串口，最后调用 `arm.move(...)` 把这一步发送给下位机执行。函数末尾的 `delay(pose.duration_ms + 150)`，目的是让当前动作基本走完之后再进入下一步，避免步骤之间相互重叠。

```
void run_pose_step(const PoseTarget& pose, uint8_t step_index)  
{  
    arm.set_torque(true);  
    arm.set_move_acc(PICK_PLACE_MOVE_ACC);  
  
    Serial.printf(  
        "[PickPlace] step=%u x=%.1f y=%.1f z=%.1f pitch=%.1f roll=%.1f claw=%.1f  
time=%u\n",  
        (unsigned int)step_index,  
        pose.x,  
        pose.y,
```

```

        pose.z,
        pose.pitch,
        pose.roll,
        pose.claw,
        pose.duration_ms
    );

    arm.move(pose.x, pose.y, pose.z, pose.pitch, pose.roll, pose.claw, pose.duration_ms);
    delay(pose.duration_ms + 150);
}

```

5. 有了单步执行函数之后，`run_pick_place_sequence()` 就负责把整个数组按顺序跑完。函数进入后先打印开始信息，然后用 `for` 循环依次取出 `kPickPlaceSequence[]` 中的每一步并交给 `run_pose_step()` 执行，全部完成后再打印结束信息并用蜂鸣器提示。

```

void run_pick_place_sequence()
{
    Serial.println("[PickPlace] sequence start");

    for (uint8_t i = 0; i < (sizeof(kPickPlaceSequence) / sizeof(kPickPlaceSequence[0])); ++i) {
        run_pose_step(kPickPlaceSequence[i], (uint8_t)(i + 1));
    }

    Serial.println("[PickPlace] sequence finished");
    arm.board.buzzer.set(60, 60, 2, 2200);
}

```

6. 动作在执行过程中，位姿反馈仍然是靠回包来刷新。`parse_pose_from_packet()` 负责把下位机返回的原始字节解码成当前末端位姿，`update_pose_from_packet()` 则把这份结果同步到 `arm.current_pose`，并在收到实时坐标回包时更新时间戳。

```

void update_pose_from_packet(PacketTypeDef* rx_packet)
{
    parse_pose_from_packet(rx_packet, arm.current_pose);
    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS) {
        last_servo_feedback_ms = millis();
    }
}

```

7. 主循环里持续处理 AT32 回包、按键更新、蜂鸣器刷新、电池状态刷新和 OLED 显示；当检测到 `BOOT` 键点击时，就调用 `run_pick_place_sequence()` 启动整套定点夹取搬运流程。

```

if (now >= button_enable_ms && arm.board.button.is_clicked(0)) {
    run_pick_place_sequence();
}

if (now - last_oled_refresh_ms >= OLED_REFRESH_INTERVAL_MS) {
    last_oled_refresh_ms = now;
    arm.board.oled.show_status(
        arm.current_pose.x,

```

```
    arm.current_pose.y,  
    arm.current_pose.z,  
    arm.current_pose.pitch,  
    arm.current_pose.roll,  
    arm.current_pose.claw  
);  
}
```